Nama : Elsa Marthalinda

NIM : 254107020204

Kelas : TI – 1G

LAPORAN JOBSHEET 9

2.1 Percobaan 1: Mahasiswa Mengumpulkan Tugas

Kode Program Mahasiswa7.java;

```
package Jobsheet9;
public class Mahasiswa7 {
    String nama;
    String nim;
    String kelas;
    int nilai;

    Mahasiswa7(String nim, String nama, String kelas) {
        this.nama = nama;
        this.nim = nim;
        this.kelas = kelas;
        this.nilai = -1;
    }

    void tugasDinilai(int nilai) {
        this.nilai = nilai;
    }
}
```

StackTugasMahasiswa7.java;

```
package Jobsheet9;
public class StackTugasMahasiswa7 {
    Mahasiswa7[] stack;
```

```
int top;
int size;

public StackTugasMahasiswa7(int size) {
    this.size = size;
    stack = new Mahasiswa7[size];
    top = -1;
}

public boolean isFull() {
    if (top == size - 1) {
        return true;
    } else {
        return false;
    }
}

public boolean isEmpty() {
    if (top == -1) {
        return true;
    } else {
        return false;
    }
}

public void push(Mahasiswa7 mhs) {
    if (!isFull()) {
        top++;
        stack[top] = mhs;
    } else {
        System.out.println("Stack penuh! Tidak bisa menambahkan tugas lagi.");
    }
```

```
    }

    public Mahasiswa7 pop() {
        if (!isEmpty()) {
            Mahasiswa7 m = stack[top];
            top--;
            return m;
        } else {
            System.out.println("Stack kosong! Tidak ada tugas untuk dinilai.");
            return null;
        }
    }

    public Mahasiswa7 peek() {
        if (!isEmpty()) {
            return stack[top];
        } else {
            System.out.println("Stack kosong! Tidak ada tugas untuk dikumpulkan");
            return null;
        }
    }

    public void print() {
        for (int i = 0; i <= top; i++) {
            System.out.println(stack[i].nama + "\t" +stack[i].nim + "\t" + stack[i].kelas);
        }
        System.out.println("");
    }
}
```

MahasiswaDemo7.java;

```
package Jobsheet9;
```

```
import java.util.Scanner;

public class MahasiswaDemo7 {
    public static void main(String[] args) {
        StackTugasMahasiswa7 stack = new StackTugasMahasiswa7(5);
        Scanner scan = new Scanner(System.in);
        int pilih;

        do {
            System.out.println("\nMenu:");
            System.out.println("1. Mengumpulkan Tugas");
            System.out.println("2. Menilai Tugas");
            System.out.println("3. Melihat Tugas Teratas");
            System.out.println("4. Melihat Daftar Tugas");
            System.out.print("Pilih: ");
            pilih = scan.nextInt();
            scan.nextLine();

            switch (pilih) {
                case 1:
                    System.out.print("Nama: ");
                    String nama = scan.nextLine();
                    System.out.print("NIM: ");
                    String nim = scan.nextLine();
                    System.out.print("Kelas: ");
                    String kelas = scan.nextLine();

                    Mahasiswa7 mhs = new Mahasiswa7(nim, nama, kelas);
                    stack.push(mhs);
                    System.out.printf("Tugas %s berhasil dikumpulkan\n", mhs.nama);
                    break;
```

```
            case 2:
                Mahasiswa7 dinilai = stack.pop();
                if (dinilai != null) {
                    System.out.println("Menilai tugas dari " + dinilai.nama);
                    System.out.print("Masukkan nilai (0-100): ");
                    int nilai = scan.nextInt();
                    dinilai.tugasDinilai(nilai);
                    System.out.printf("Nilai Tugas %s adalah %d\n", dinilai.nama, nilai);
                }
                break;

            case 3:
                Mahasiswa7 lihat = stack.peek();
                if (lihat != null) {
                    System.out.println("Tugas terakhir dikumpulkan oleh " + lihat.nama);
                }
                break;

            case 4:
                System.out.println("Daftar semua tugas");
                System.out.println("Nama\tNIM\tKelas");
                stack.print();
                break;
            default:
                System.out.println("Pilihan tidak valid.");
        }
    } while (pilih >= 1 && pilih <= 4);
  }
}
```

Hasil Ouput:

```
Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 1
Nama: Dila
NIM: 1001
Kelas: 1A
Tugas Dila berhasil dikumpulkan

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 1
Nama: Erik
NIM: 1002
Kelas: 1B
Tugas Erik berhasil dikumpulkan

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 3
Tugas terakhir dikumpulkan oleh Erik
```

```
Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 1
Nama: Tika
NIM: 1003
Kelas: 1C
Tugas Tika berhasil dikumpulkan

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 4
Daftar semua tugas
Nama    NIM     Kelas
Dila    1001    1A
Erik    1002    1B
Tika    1003    1C
```

```
Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 2
Menilai tugas dari Tika
Masukkan nilai (0-100): 87
Nilai Tugas Tika adalah 87

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 4
Daftar semua tugas
Nama    NIM     Kelas
Erik    1002    1B
Dila    1001    1A

Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
```

2.1.3 Pertanyaan

1. Lakukan perbaikan pada kode program, sehingga keluaran yang dihasilkan sama dengan verifikasi hasil percobaan! Bagian mana yang perlu diperbaiki?

```java
public void print() {
    for (int i = top; i >= 0; i--) {
        System.out.println(stack[i].nama + "\t" + stack[i].nim + "\t" + stack[i].kelas);
    }
}
```

➢

```
Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 4
Daftar semua tugas
Nama    NIM     Kelas
Tika    1003    1C
Erik    1002    1B
Dila    1001    1A
```

➢

```
Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
Pilih: 4
Daftar semua tugas
Nama    NIM     Kelas
Erik    1002    1B
Dila    1001    1A
```

2. Berapa banyak data tugas mahasiswa yang dapat ditampung di dalam Stack? Tunjukkan potongan kode programnya!
   - Jumlah data tugas mahasiswa yang dapat ditampung di dalam stack ditentukan saat pembuatan objek stack melalui konstruktor. Pada program ini, kapasitas stack diinisialisasi sebesar 5, sehingga stack hanya mampu menyimpan maksimal 5 data mahasiswa. Hal ini terlihat pada saat objek stack dibuat dengan parameter 5. Dengan demikian, jika jumlah data yang dimasukkan sudah mencapai batas tersebut, maka stack tidak dapat menerima data tambahan lagi.

```
StackTugasMahasiswa7 stack = new StackTugasMahasiswa7(size: 5);
```

3. Mengapa perlu pengecekan kondisi !isFull() pada method push? Kalau kondisi if-else tersebut dihapus, apa dampaknya?
   - Pengecekan kondisi !isFull() pada method push bertujuan untuk memastikan bahwa stack masih memiliki ruang kosong sebelum menambahkan data baru. Jika kondisi ini tidak dilakukan, maka program akan tetap mencoba menambahkan data ke dalam array meskipun kapasitasnya sudah penuh. Hal ini dapat menyebabkan terjadinya error berupa *ArrayIndexOutOfBoundsException*, karena program mengakses indeks array di luar batas yang tersedia.
4. Modifikasi kode program pada class MahasiswaDemo dan StackTugasMahasiswa sehingga pengguna juga dapat melihat mahasiswa yang pertama kali mengumpulkan tugas melalui operasi lihat tugas terbawah!

```
Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
5. Melihat Tugas Terbawah
Pilih: 5
Tugas pertama dikumpulkan oleh Dila
```

5. Tambahkan method untuk dapat menghitung berapa banyak tugas yang sudah dikumpulkan saat ini, serta tambahkan operasi menunya!

```
public int jumlahTugas() {
    return top + 1;
}
```

```
System.out.println("6. Jumlah Tugas");
```

```
case 6:
    System.out.println("Jumlah tugas saat ini: " + stack.jumlahTugas());
    break;
```

```
Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
5. Melihat Tugas Terbawah
6. Jumlah Tugas
Pilih: 6
Jumlah tugas saat ini: 2
```

6. **Commit dan push kode program ke Github**

2.2 Percobaan 2: Konversi Nilai Tugas ke Biner

Kode Program StacKonversi7.java;

```
package Jobsheet9;
public class StackKonversi7 {
    int[] tumpukanBiner;
    int size;
    int top;

    public StackKonversi7() {
        this.size = 32; //asumsi 32 bit
        tumpukanBiner = new int [size];
        top = -1;
    }
    public boolean isEmpty() {
        return top == -1;
    }
    public boolean isFull() {
```

```
        return top == size -1;
    }
    public void push(int data) {
        if (isFull()) {
            System.out.println("Stack penuh");
        } else {
            top++;
            tumpukanBiner[top] = data;
        }
    }
    public int pop() {
        if (isEmpty()) {
            System.out.println("Stack kosong");
            return -1;
        } else {
            int data = tumpukanBiner[top];
            top--;
            return data;
        }
    }
}
```

StackTugasMahasiswa7.java;

```
package Jobsheet9;
public class StackTugasMahasiswa7 {
    Mahasiswa7[] stack;
    int top;
    int size;

    public StackTugasMahasiswa7(int size) {
        this.size = size;
        stack = new Mahasiswa7[size];
```

```
    top = -1;
}

public boolean isFull() {
    if (top == size -1) {
        return true;
    } else {
        return false;
    }
}

public boolean isEmpty() {
    if (top == -1) {
        return true;
    } else {
        return false;
    }
}

public void push(Mahasiswa7 mhs) {
    if (!isFull()) {
        top++;
        stack[top] = mhs;
    } else {
        System.out.println("Stack penuh! Tidak bisa menambahkan tugas lagi.");
    }
}

public Mahasiswa7 pop() {
    if (!isEmpty()) {
        Mahasiswa7 m = stack[top];
        top--;
```

```
        return m;
    } else {
        System.out.println("Stack kosong! Tidak ada tugas untuk dinilai.");
        return null;
    }
}

public Mahasiswa7 peek() {
    if (!isEmpty()) {
        return stack[top];
    } else {
        System.out.println("Stack kosong! Tidak ada tugas yang dikumpulkan");
        return null;
    }
}

public void print() {
    for (int i = top; i >= 0; i--) {
        System.out.println(stack[i].nama + "\t" + stack[i].nim + "\t" + stack[i].kelas);
    }
}

public Mahasiswa7 peekBottom() {
    if (!isEmpty()) {
        return stack[0];
    } else {
        System.out.println("Stack kosong!");
        return null;
    }
}

public int jumlahTugas() {
```

```
        return top + 1;
    }

    public String konversiDesimalKeBiner(int nilai) {
        StackKonversi7 stack = new StackKonversi7();
        while (nilai > 0) {
            int sisa = nilai % 2;
            stack.push(sisa);
            nilai = nilai / 2;
        }
        String biner = new String();
        while (!stack.isEmpty()) {
            biner += stack.pop();
        }
        return biner;
    }
}
```

MahasiswaDemo7.java;

```
package Jobsheet9;
import java.util.Scanner;

public class MahasiswaDemo7 {
    public static void main(String[] args) {
        StackTugasMahasiswa7 stack = new StackTugasMahasiswa7(5);
        Scanner scan = new Scanner(System.in);
        int pilih;

        do {
            System.out.println("\nMenu:");
            System.out.println("1. Mengumpulkan Tugas");
            System.out.println("2. Menilai Tugas");
```

```
System.out.println("3. Melihat Tugas Teratas");
System.out.println("4. Melihat Daftar Tugas");
System.out.println("5. Melihat Tugas Terbawah");
System.out.println("6. Jumlah Tugas");
System.out.print("Pilih: ");
pilih = scan.nextInt();
scan.nextLine();

switch (pilih) {
    case 1:
        System.out.print("Nama: ");
        String nama = scan.nextLine();
        System.out.print("NIM: ");
        String nim = scan.nextLine();
        System.out.print("Kelas: ");
        String kelas = scan.nextLine();
        Mahasiswa7 mhs = new Mahasiswa7(nama, nim, kelas);
        stack.push(mhs);
        System.out.printf("Tugas %s berhasil dikumpulkan\n", mhs.nama);
        break;

    case 2:
        Mahasiswa7 dinilai = stack.pop();
        if (dinilai != null) {
            System.out.println("Menilai tugas dari " + dinilai.nama);
            System.out.print("Masukkan nilai (0-100): ");
            int nilai = scan.nextInt();
            dinilai.tugasDinilai(nilai);
            System.out.printf("Nilai Tugas %s adalah %d\n", dinilai.nama, nilai);
            String biner = stack.konversiDesimalKeBiner(nilai);
            System.out.printf("Nilai Dalam Biner: " + biner);
        }
```

```
                break;

            case 3:
                Mahasiswa7 lihat = stack.peek();
                if (lihat != null) {
                    System.out.println("Tugas terakhir dikumpulkan oleh " + lihat.nama);
                }
                break;

            case 4:
                System.out.println("Daftar semua tugas");
                System.out.println("Nama\tNIM\tKelas");
                stack.print();
                break;
            case 5:
                Mahasiswa7 bawah = stack.peekBottom();
                if (bawah != null) {
                    System.out.println("Tugas pertama dikumpulkan oleh " + bawah.nama);
                }
                break;
            case 6:
                System.out.println("Jumlah tugas saat ini: " + stack.jumlahTugas());
                break;
            default:
                System.out.println("Pilihan tidak valid.");
        }
    } while (pilih >= 1 && pilih <= 6);
  }
}
```

Hasil Output MahasiswaDemo7.java

```
Menu:
1. Mengumpulkan Tugas
2. Menilai Tugas
3. Melihat Tugas Teratas
4. Melihat Daftar Tugas
5. Melihat Tugas Terbawah
6. Jumlah Tugas
Pilih: 2
Menilai tugas dari Tika
Masukkan nilai (0-100): 87
Nilai Tugas Tika adalah 87
Nilai Dalam Biner: 1010111
```

2.2.3 Pertanyaan

1. Jelaskan alur kerja dari method konversiDesimalKeBiner!
   - Method ini bekerja dengan cara mengubah bilangan desimal menjadi bilangan biner menggunakan bantuan struktur data stack. Proses dimulai dengan membagian nilai dengan 2 secara berulang. Pada setiap proses pembagian, sisa hasil bagi (yang bernilai 0 atau 1) akan dimasukkan ke dalam stack. Setelah itu, nilai diperbarui dengan hasil pembagian tersebut hingga akhirnya menjadi nol. Setelah semua sisa pembagian dimasukkan ke dalam stack, langkah berikutnya adalah mengambil kembali isi stack satu per satu. Karena stack bersifat LIFO (Last In First Out). Proses ini secara otomatis membalik urutan sisa pembagian, sehingga menghasilkan susunan angka biner yang benar. Setiap elemen yang di-pop dari stack kemudian digabungkan ke dalam sebuah string hingga stack kosong. Pada akhirnya, string yang terbentuk merupakan hasil konversi bilangan desimal ke biner, dan nilai tersebut dikembalikan oleh method sebagai output.
2. Pada method konversiDesimalKeBiner, ubah kondisi perulangan menjadi while (kode != 0), bagaimana hasilnya? Jelaskan alasannya!
   - Apabila kondisi perulangan diubah menjadi while (nilai != 0), maka untuk bilangan positif hasilnya akan tetap sama. Hal ini karena proses pembagian akan terus berlangsung sampai nilai menjadi 0, sehingga perulangan berhenti secara otomatis. Namun, penggunaan kondisi while (nilai != 0) kurang tepat dibandingkan while (nilai > 0). Hal ini karena kondisi tersebut juga memungkinkan perulangan berjalan untuk nilai negatif, yang sebenarnya tidak sesuai dengan konsep konversi bilangan desimal ke biner dalam konteks ini. Jika nilai negatif dimasukkan, hasil yang diperoleh bisa tidak valid atau menyebabkan perilaku yang tidak diinginkan.